esiea

# ANGULAR

## Les bases

Ressources :

https://angular.io/docs

# Préambule

esiea

- À partir de 2005, le système AJAX (Asynchronous Javascript And XML) permet des interactions entre l’utilisateur et des backend HTTP : il est enfin possible d'échanger des informations et de générer du contenu à partir de ces interactions.

- Cette bibliothèque permet notamment de rafraîchir une seul partie d’une page sans recharger la page entière.

- En 2010, la première version d’AngularJS est lancée. Elle permet de créer plus facilement des Single Page Applications, des applications web qui imitent les applications

# Préambule

esiea

- Cette version permet déjà de faire énormément de choses, mais souffre d'une syntaxe plutôt complexe ainsi que des limitations du JavaScript.

- Voilà pourquoi Google choisit de complètement réécrire le framework pour sa version 2. Aujourd’hui, nous en sommes à Angular 21.x (maintenant appelé simplement “Angular”)

# Qu’est-ce qu’un « Framework »

esiea

- Un framework est une bibliothèque de fonctions pré-écrites facilitant le développement d'applications en fournissant des services pré-codés et testés afin que le développeur puisse se concentrer sur les fonctionnalités métier plutôt que sur des problématiques techniques.

- Angular est un framework javascript facilitant le développement d'applications fondées sur JavaScript, et en particulier pour Ajax et autres applications centrées Web.

# Pourquoi Angular ?

esiea

- Les avantages d'utiliser Angular par rapport à un front sans framework ou à d'autres frameworks sont les suivants :
  - pas de rechargement de la page HTML complète ce qui réduit les temps de chargement
  - Angular est géré par Google : il y a donc peu de chances qu'il disparaisse, et l'équipe de développement du framework est excellente.
  - Le TypeScript : ce langage permet un développement beaucoup plus stable, rapide et facile.
  - Le framework Ionic (le framework permettant le développement d'applications mobiles multi-plateformes à partir d'une seule base de code) utilise Angular.
  - Complète compatibilité avec Firebase, le back-end fournit par Google

# TypeScript

esiea

- En bref, TypeScript est un sur-ensemble (superset) de JavaScript transcompilé en JavaScript (le moteur Angular transforme le TypeScript en JavaScript avant de l'envoyer au serveur) pour être compréhensible par les navigateurs.
- Il ajoute à JavaScript de nombreuses fonctionnalités utiles comme :
  - La prise en compte de la POO
  - Le typage des variables.

# Installation

esiea

- Comment installer Angular sur son PC ?
- Le CLI, ou “Command Line Interface” (un outil permettant d'exécuter des commandes depuis la console) d’Angular est l’outil qui vous permet d’exécuter des scripts pour la création, la structuration et la production d’une application Angular.
- Il utilise, pour son installation les outils suivants :
  - **`NodeJs`** que vous avez déjà utilisé au premier semestre et qui doit être toujours présent sur votre PC. Sinon, réinstallez-le.
  - **`Npm`** (Node Packet Manager) qui est le gestionnaire de paquets de NodeJs permettant d’installer ou supprimer des « paquets » ajoutant des fonctionnalités à NodeJs en gérant les versions.

# Installation

esiea

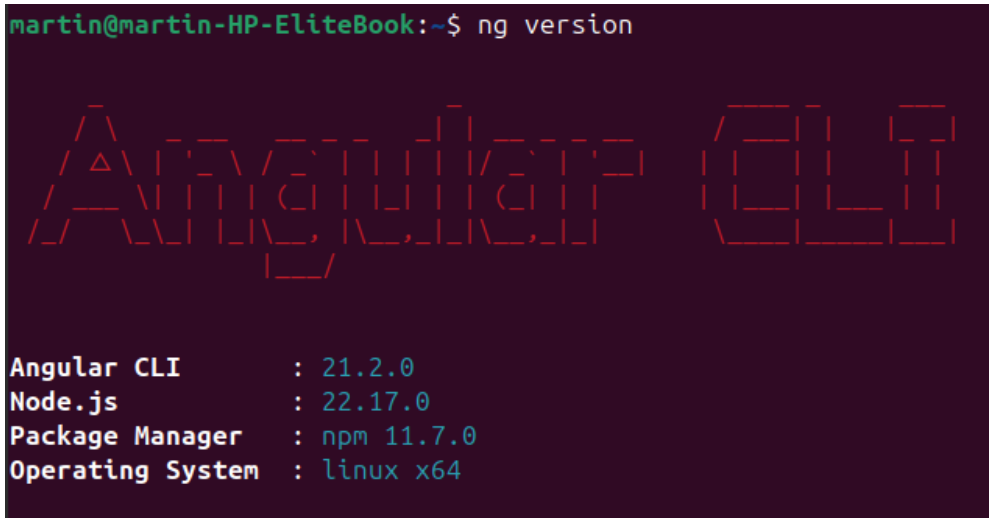
- Pour installer (ou mettre à jour) npm, tapez la commande suivante dans l'invite de commande :

```
npm install -g
npm@latest
```

- Pour installer Angular, tapez la commande suivante dans l'invite de commande :

```
npm install -g @angular/cli
```

- Pour vérifier que l'installation s'est bien passée, tapez la commande suivante dans l'invite de commande : `ng version`



# Création d'un projet Angular

esiea

- Pour créer un nouveau projet Angular :
  - Créez ou rendez-vous dans le dossier parent de votre futur projet Angular ;
  - Depuis l'invite de commande, saisissez la commande suivante : `ng new mon-projet` en remplaçant, bien entendu « **`mon-projet`** » par le nom de votre projet.
- Lorsqu'on vous demande quelle type de feuille de style utiliser, choisissez « Sass (SCSS) ».
- Lorsqu'on vous demande si vous voulez activer le « SSR », répondez « No ».
- De même, refusez d'ajouter un fichier de configuration IA au projet.

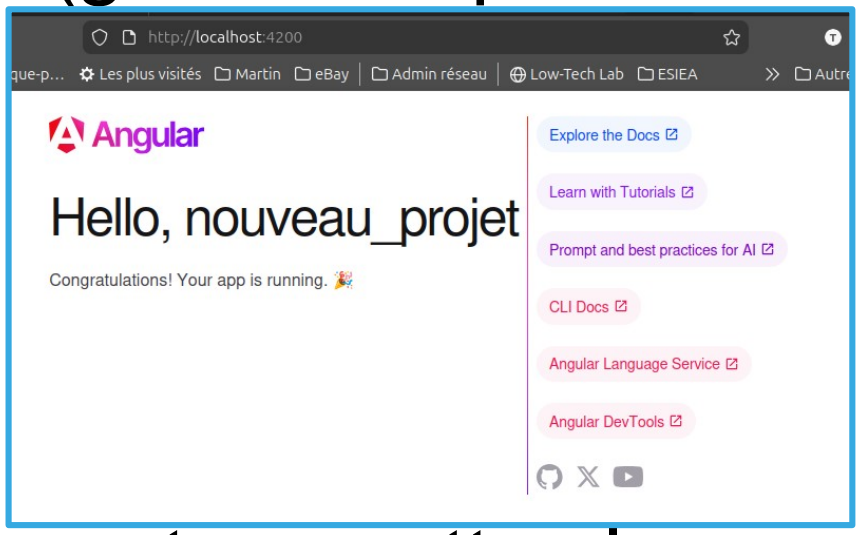
# Création d'un projet Angular

esiea

- Ensuite, naviguez dans le dossier du projet et lancez le serveur de développement :

```
cd mon-projet
ng serve -o
```

- Votre navigateur par défaut ouvre normalement votre projet en local sur le port 4200 (grâce à l'option « -o »).

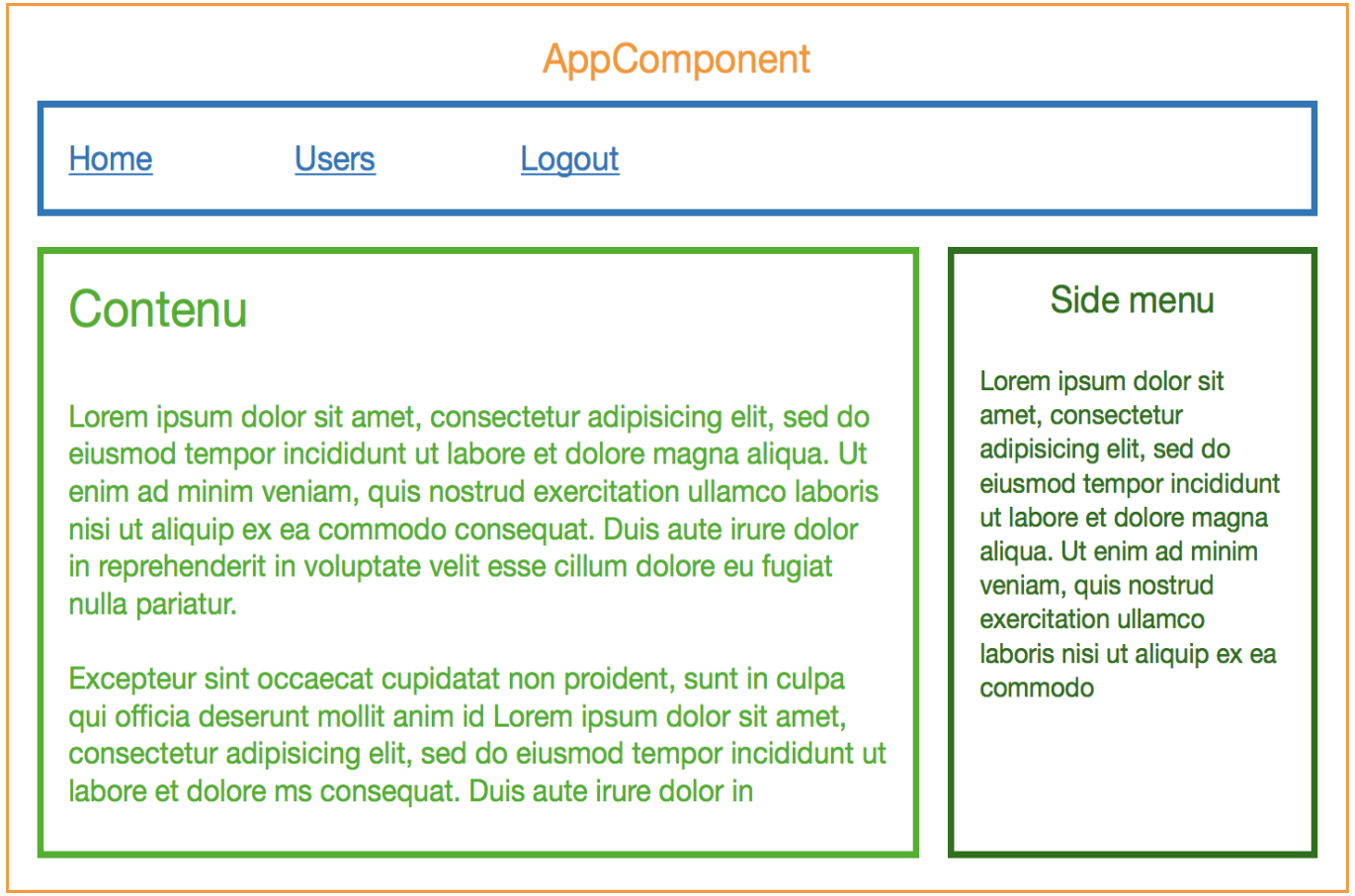



- Pour de futurs lancements, omettez le « -o » pour éviter d'ouvrir un nouvel onglet.

# Structure d'une page Angular

esiea

- Une page Angular est hiérarchisée en composants (« **`Components`** ») qui se contiennent les uns, les autres.



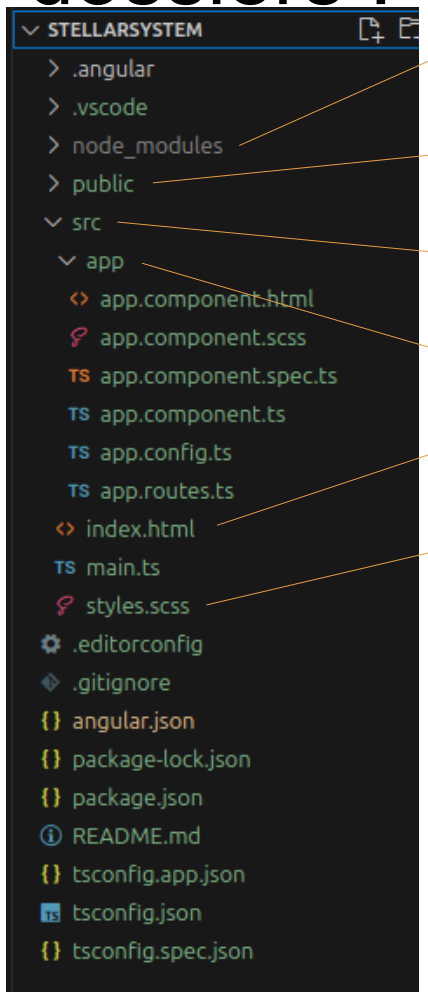

**`AppComponent`** est le composant principal

Les autres composants seront imbriqués dans celui-ci.

# Structure d'un projet Angular

esiea

- Le projet créé par Angular contient de nombreux fichiers et dossiers :




Dossier contenant tous les modules (librairies externes) ajoutées au projet

Dossier contenant tous les fichiers publics du projet (images, icones, ...)

Dossier contenant les sources du projet

Dossier contenant le component principal du projet

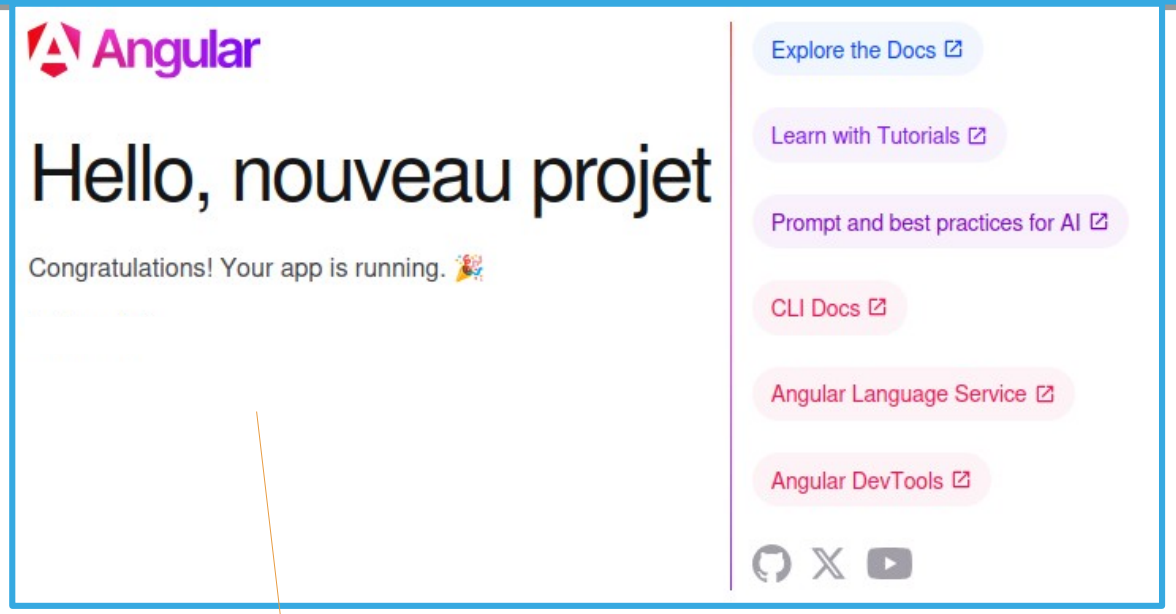
Unique **page** html du projet

Feuille de style principale du projet

# Comment ça fonctionne ?

esiea

- Si l'on compare la source **`index.html`** et ce qu'affiche notre navigateur, il y a de quoi se poser des questions !




```
src > index.html > ...
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>StellarSystem</title>
6    <base href="/">
7    <meta name="viewport" content="width=device-width, initial-scale=1">
8    <link rel="icon" type="image/x-icon" href="favicon.ico">
9  </head>
10 <body>
11   <app-root></app-root>
12 </body>
13 </html>
```

D'où viennent toutes ces images et tout ce texte alors que la balise **`body`** de l'HTML ne contient qu'une balise étrange (et vide) **`app-root`** ?

# app-root est un component

esiea

- La balise **`app-root`** présent dans le **`body`** est une balise spécifique à angular spécifiant qu'à cet endroit de la page, on doit afficher le component « **`app`** ».

- Or, si on regarde le fichier source **`app/app.html`**, on y découvre le code qui explique le contenu de notre page affichée sur le navigateur.

- Comment Angular injecte-t-il ce code dans la balise **`app-root`** ?

# Anatomie d'un component

esiea

- Regardons le contenu du fichier **`app.ts`** :

```
import { Component,signal} from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrl: './app.scss'
})
export class App {
  protected readonly title = signal('nouveau_projet');
}
```

Annotation **`@Component`** spécifiant que la classe définie dessous est un composant.

Paramètre **`selector`** spécifiant la balise HTML spécifique de ce composant.

Import(s) spécifique(s) nécessaire(s) au bon fonctionnement du composant.

Paramètre **`templateUrl`** spécifiant le chemin relatif vers le source HTML de ce composant.

Paramètre **`styleUrl`** spécifiant le chemin relatif vers la feuille de style de ce composant.

Définition d'une constante d'affichage sous la forme d'un signal (vu plus tard).

# Anatomie d'un component

esiea

- Pour affiner notre compréhension de la structure d'un composant, nous allons en créer un nouveau.
- On utilise de nouveau la commande ng pour cela :

```
ng generate component <chemin>NomComposant
```

- Où :
  - **`Chemin`** est le chemin relatif du dossier dans lequel sera créé le composant par rapport au dossier **`src/app`**.
  - **`NomComposant`** est le nom du composant (auquel Angular rajoutera « **`-component`** »).
- Il existe une version abrégée de cette commande :

```
ng g c <chemin>NomComposant
```

# Anatomie d'un component

esiea

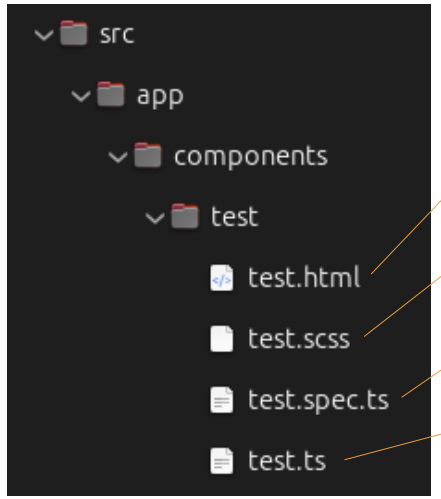
- Pour prendre de bonnes habitudes dès le départ créons notre composant dans un dossier approprié :

```
ng g c components/test
```

- Résultat en ligne de commande :

```
CREATE src/app/components/test/test.scss (0 bytes)
CREATE src/app/components/test/test.component.html (19 bytes)
CREATE src/app/components/test/test.spec.ts (517 bytes)
CREATE src/app/components/test/test.ts (179 bytes)
```

- Résultat sur la structure du projet :




Fichier template HTML du composant

Feuille de style du composant

Script de test unitaire du composant

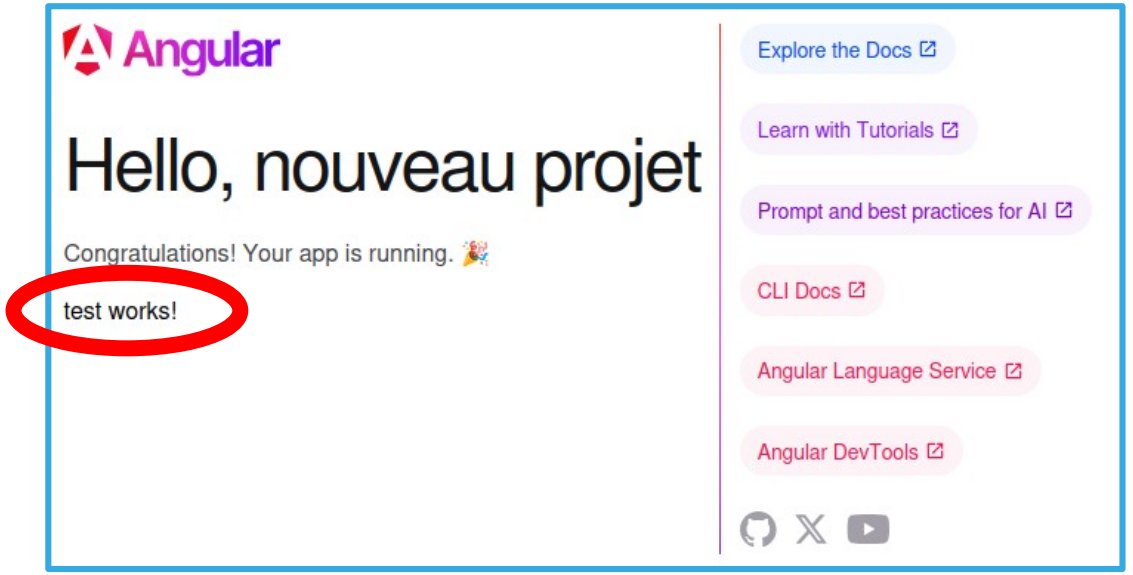
Source de la classe du composant

# Anatomie d'un component

esiea

- Pour afficher notre composant, il suffit maintenant :
  - D'aller récupérer le nom de sa balise dans son **`.ts`** (ici « **`app-test`** »).
  - De rajouter cette balise dans l'HTML du composant principal « **`app.html`** ».
  - De rajouter la classe **`Test`** dans les imports de la classe **`App`**.
  - Résultat :

**Note :** si vous utilisez l'autocomplétion de VSCode pour ajouter la balise dans l'HTML, l'import dans le .ts sera fait automatiquement, mais il faudra bien vérifier que la sauvegarde du fichier est faite sinon, lors du lancement, vous aurez une erreur « NG8001 »



# Databinding

esiea

- La partie TypeScript du composant Angular permet de faire des calculs, accéder à un back-end, … et de mettre à disposition du HTML les résultats obtenus.
- Il existe plusieurs mécanismes pour cela :
  - **String interpolation** : afficher directement la valeur d'une variable TS dans l'HTML
  - **Property binding** : modifier les attributs d'une balise HTML à partir du contenu d'une variable TS.
  - **Event Binding** : réagir à un évènement HTML (clic, clavier, …) en appelant une méthode TS.
  - **Form Binding** : liaison entre les champs d'un formulaire HTML et un modèle objet en TS.

# String interpolation

esiea

- Regardez la source **`app.ts`**. Dans la classe, la seule ligne présente est :

```
protected readonly title = signal('nouveau projet');
```

- Commençons par prendre la bonne habitude de typer nos variables. On en profite pour enlever ce signal inutile ici :

```
protected readonly title : string = 'nouveau projet';
```

- Si on regarde maintenant **`app.html`** aux alentours de la ligne 230, on trouve :

```
<h1>Hello, {{ title() }}</h1>
<p>Congratulations! Your app is running. 🎉</p>
```

Il faut enlever les « `()` » vu que **`title`** n'est plus un signal

- Or, notre page, dans le navigateur affiche :

Hello, nouveau projet

Congratulations! Your app is running. 🎉

- Vous l'aurez compris, la syntaxe **`{{ variable }}`** dans un fichier HTML affiche sur la page le contenu de la variable **`variable`** définie dans la classe.

# String interpolation

esiea

- On peut utiliser aussi l'interpolation de chaîne pour afficher le retour d'une méthode :
  - Définissez dans le fichier **`app.ts`** une méthode qui renvoie l'URL du serveur sur lequel tourne votre Angular.
    Elle peut être obtenue par la ligne suivante :

```
url : String = `${window.location.protocol}//${window.location.hostname}:${window.location.port}`;
```

  - Créez une méthode renvoyant cette URL.
  - Utilisez l'interpolation de chaîne pour afficher la sortie de la méthode dans l'HTML sous cette forme :

Hello, nouveau projet

Congratulations! Your app is running
from url http://localhost:4200. 🎉

# String interpolation

esiea

- Voici les modifications dans le TS :

```
export class App {
  title : String = 'nouveau projet';
  url : String = `${window.location.protocol}//${window.location.hostname}:${window.location.port}`;

  getUrl() : String {
    return this.url;
  }
}
```

- Et voici celles dans l'HTML :

```
<h1>Hello, {{ title }}</h1>
<p>Congratulations! Your app is running<br/>from URL {{ getUrl()}} 🎉</p>
```

# Property binding

esiea

- Comme écrit précédemment, ce mécanisme permet de modifier les attributs d'une balise HTML à partir du contenu d'une variable TS.
- Par exemple, on peut charger la source d'une image depuis une variable TS.

```
export class Test {
  imgUrl : String = "https://static0.gamerantimages.com/wordpress/wp-content/uploads/2022/11/Rick-with-Portals.jpg";
}
```

- dans test.html :
- Résultat :

```
<p>test works !!!</p>
<img [src]="imgUrl" width="200px">
```

Notez la syntaxe à base de crochets pour différencier la propriété « standard » du property binding..

# Event binding

esiea

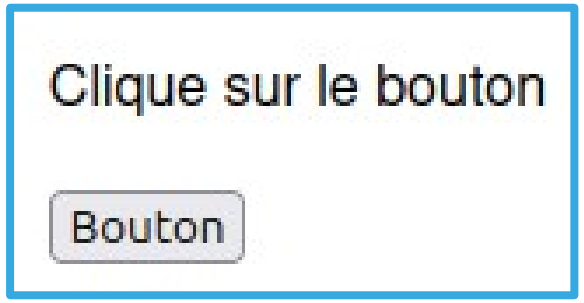
- Comme écrit précédemment, cela permet de réagir à un évènement HTML (clic, clavier, …) en appelant une méthode TS.
- Par exemple, on peut afficher une popup lors d'un clic sur un bouton.
  - Dans **`test.ts`** :

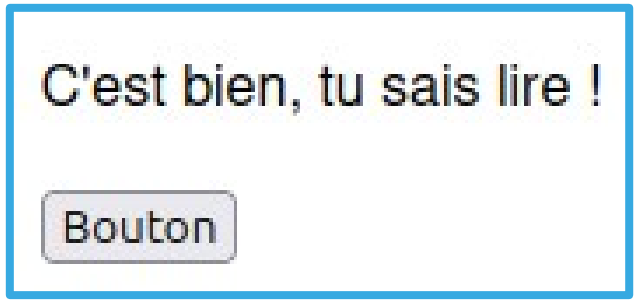
```
export class Test {
  readonly label : WritableSignal<string> =
                   signal("Clique sur le bouton");
  onClic() : void {
    this.label.set("C'est bien, tu sais lire !");
  }
}
```

  - dans **`test.html`** :

```
<p>{{label()}}</p>
<button (click)="onClic()">Bouton</button>
```

  - Résultat (avant/après) :




Notez la syntaxe à base de parenthèses pour différencier l'évènement « standard » de l'event binding.

# Property et event binding

esiea

- Créez un component qui contient un bouton et une image. Par défaut :
  - Le bouton affiche le texte « Allumer »
  - L'image représente un voyant éteint
- Quand on clique sur le bouton :
  - Le texte du bouton devient « Éteindre »
  - L'image représente un voyant allumé
- Vous pouvez utiliser les images situées aux URL suivantes :
  - http://www.i2clipart.com/cliparts/f/a/e/0/clipart-voyant-rouge-eteint-red-light-off-128x128-fae0.png
  - http://www.i2clipart.com/cliparts/8/4/3/b/clipart-voyant-rouge-allume-red-light-on-128x128-843b.png

# Property et event binding

esiea

- On commence par créer quelques constantes dans le .ts :

```
const ON_URL : string = "http://www.i2clipart.com/cliparts/8/4/3/b/clipart-voyant-rouge-allume-red-light-on-128x128-843b.png";
const OFF_URL : string = "http://www.i2clipart.com/cliparts/f/a/e/0/clipart-voyant-rouge-eteint-red-light-off-128x128-fae0.png";
const OFF_BTN : string = "Allumer";
const ON_BTN : string = "Eteindre";
```

- Puis la classe :

```
export class Light {
  protected readonly lightSource : WritableSignal<string> = signal(OFF_URL);
  protected readonly texteBouton : WritableSignal<string> = signal(OFF_BTN);
  allume : boolean = false;

  switchLight() : void {
    if (this.allume) {
      this.lightSource.set(OFF_URL);
      this.texteBouton.set(OFF_BTN);
      this.allume = false;
    } else {
      this.lightSource.set(ON_URL);
      this.texteBouton.set(ON_BTN);
      this.allume = true;
    }
  }
}
```

Ici, **`texteBouton`** et **`lightSource`** ayant vocation à changer régulièrement et ayant un impact sur l'affichage, il est intéressant de passer par des signaux.

Property binding.

Event binding.

String interpolation

- Et voici l'affichage :

```
<img [src]="lightSource()">
<button (click)="switchLight()">{{texteBouton()}}</button>
```

# Les directives

esiea

- Angular possède trois types de directives afin de faciliter la vie du développeur :
  - Les directives structurelles permettent de rajouter des éléments de programmation (test, bouclage, ...) au sein de la structure HTML ;
  - Les directives par attribut permettent de jouer sur le style d'un élément en jouant sur ses attributs **style** et **class** ;
  - Les pipes permettent de chaîner des transformations de texte avant l'affichage pour, par exemple, le formater

# Directives structurelles

esiea

- Il existe deux types de syntaxe pour les directives structurelles :
  - Les directives « ancienne génération » sont des directives conçues comme des attributs d'une balise HTML et se reconnaissent par leur préfixe « **`*ng`** » ;
  - Les directives « nouvelle génération » s'insèrent entre deux balises HTML (comme les directives php) et se reconnaissent par leur préfixe « `@` »
- L'ancienne syntaxe vous est donnée si vous aviez à maintenir du code ancien, mais il faudra toujours préférer la nouvelle, plus optimisée.

# Directive structurelle : if

esiea

- Voici les deux façons de n'afficher un paragraphe que si la variable ts « **`allume`** » (transformée en signal pour l'occasion) est vraie :

```
@if (allume()) {
    <p>La lumière est allumée.</p>
} @else {
    <p>La lumière est éteinte.</p>
}
<p *ngIf="allume()" >La lumière est allumée.</p>
```

La nouvelle syntaxe permet le « **@else if** » et le « **@else** »

L'ancienne ne permet ni le « **@else if** » ni le « **@else** »

- Contrairement à la nouvelle syntaxe, totalement intégrée au moteur Angular, l'ancienne demande l'import de « **`CommonModule`** » dans le component (section **`imports`** du **`.ts`**). Sinon, elle ne sera pas reconnue.

# Directive structurelle : for

esiea

- Ajoutons un tableau d'utilisateurs dans notre classe :

```
users = [
  {
    id:9,
    nom: 'Dupont',
    prenom: 'Antoine'
  },
  {
    id:10,
    nom: 'Ntamack',
    prenom: 'Romain'
  },
  {
    id:15,
    nom: 'Ramos',
    prenom: 'Thomas'
  }
]
```

Le tableau n'est pas typé pour le moment, nous verrons plus tard comment faire.

# Directive structurelle : for

esiea

- Pour afficher ce tableau via l'ancienne directive (ne pas oublier l'import de « **`CommonModule`** » :

Variable contenant l'élément courant de la boucle..

Le tableau défini dans la slide d'avant.

```
<p *ngFor="let user of users" >{{user.prenom}} {{user.nom}}
porte le numéro {{user.id}}</p>
```

- Résultat:

Antoine Dupont porte le numéro 9

Romain Ntamack porte le numéro 10

Thomas Ramos porte le numéro 15

- Cela fonctionne mais n'est pas optimisé et ne gère pas le cas où le tableau est vide.

# Directive structurelle : for

esiea

- Pour afficher ce tableau via la nouvelle syntaxe (avec le même résultat) :

Le **`track`** est obligatoire. Il sert à faire le lien entre l'objet affiché et le DOM. Si aucun id n'existe pour l'objet, on peut utiliser « **`$index`** » qui représente l'index de l'objet dans le tableau.

```
@for (user of users; track user.id) {
    <p>{{user.prenom}} {{user.nom}} porte le numéro {{user.id}}</p>
} @empty {
    <p>l'équipe est vide.</p>
}
```

Cette syntaxe gère le cas du tableau vide via la directive « **`@empty`** » (qui doit être juste après la fermeture du bloc « **`@for`** »).

- Cette syntaxe permet l'utilisation des variables suivantes dans le bloc : **`@count`**, **`@index`**, **`@first`**, **`@last`**, **`@even`** et **`@odd`** qui peuvent être très utiles dans la mise en forme des listes.

# Directive structurelle : switch

esiea

- Cette directive n’existe que dans la nouvelle syntaxe. Voici un exemple qui permet d’afficher un component différent selon le profil de l’utilisateur :

```
@switch (userPermissions) {
    @case ('admin') {
      <app-admin-dashboard />
    }
    @case ('reviewer') {
      <app-reviewer-dashboard />
    }
    @case ('editor') {
      <app-editor-dashboard />
    }
    @default {
      <app-viewer-dashboard />
    }
}
```

# Directive attribut : *ngStyle

esiea

- Le binding d'attribut permet de modifier le style d'une balise en fonction d'une variable ts :

```
<div [style.color]="errorMessageColor">
   Le numéro de téléphone n'est pas au bon format.
</div>
```

- Cependant, ce binding ne gère que des styles simples. **Pour des styles plus compliqués**, on utilisera :

```
<div [ngStyle]="currentStyles">
   Le numéro de téléphone n'est pas au bon format.
</div>
```

- Dans le ts :

```
  currentStyle = {
    'font-style':  this.canSave  ? 'italic' : 'normal',
    'color':       this.hasError ? 'red'    : 'black',
    'font-size':   this.hasError ? '24px'   : '12px'
}
```

# Directive attribut : *ngClass

esiea

- La directive **`*ngClass`** permet d'associer une ou plusieurs classes CSS à une balise en fonction de conditions :

Les accolades ne sont nécessaires que si on veut affecter plusieurs classes CSS.

```
<li [ngClass]="{
        'list-group-item':true,
        'list-group-item-success': allume,
        'list-group-item-danger': !allume}">
    Un item de liste qui s'allume et s'éteint.
</li>
```

- Ici, notre item de liste aura toujours la classe CSS **`list-group-item`** mais la deuxième classe dépendra de la valeur de la variable **`allume`**.

# Le pipes

esiea

- Les pipes prennent des données en input, les transforment, et affichent ensuite les données modifiées dans le DOM.
- Des pipes sont fournis avec Angular mais il est également possible d'en créer au besoin.

- Elles sont utilisées (en général dans une interpolation de chaîne) avec la syntaxe suivante :

```
{{nomvar | format1 | format2}}
```

On peut chaîner autant de formateurs que voulu.

Le contenu de la variable **nomvar** est modifié par le 1er formateur.

Puis, la sortie du 1er formateur est modifié par le 2e formateur avant d'être affichée.

# Les pipes Angular

esiea

- Pour afficher la date système en angular, on peut utiliser dans le ts :

```
today : Date = new Date();
```

- Puis,dans l'HTML :

```
<p>Nous sommes aujourd'hui le {{today}}</p>
```

- Mais le résultat n'est pas fantasique :

Nous sommes aujourd'hui le Tue Mar 04 2025 12:15:06 GMT+0100 (heure normale d'Europe centrale)

- Alors que si on utilise le pipe Date :

```
<p>Nous sommes aujourd'hui le {{today | date : 'dd MMMM yyyy'}}</p>
```

Nous sommes aujourd'hui le 04 March 2025

Il faut importer `CommonModule` dans le `.ts` pour que le pipe fonctionne.

- La date est toujours en anglais. Voyons comment imposer les formats de dates, monnaies, etc en français

# passer la locale en Français

esiea

- Pour modifier la « locale » d'Angular, modifiez ainsi le **app.ts** :

```
import { Component } from '@angular/core';
import { LOCALE_ID } from '@angular/core';
import { registerLocaleData } from '@angular/common';
import frenchLocale from '@angular/common/locales/fr';
import { LightComponent } from "./components/light/light.component";

@Component({
  selector: 'app-root',
  imports: [LightComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.scss',
  providers: [{provide: LOCALE_ID, useValue: 'fr-FR' }]
})
export class AppComponent {
  title : String = 'StellarSystem';
  constructor() {
    registerLocaleData(frenchLocale);
  }
}
```

On change la valeur de la variable globale d'Angular définissant la locale utilisée.

Pour que cela fonctionne, il faut avoir importé et enregistré celle-ci depuis les librairies du framework.

# Les pipes Angular

esiea

- Une fois la locale corrigée, l'affichage des dates est adapté :

Nous sommes aujourd'hui le 04 mars 2025

- De la même façon, la locale va agir sur l'affichage des monnaies :

```
<p>le total de votre panier est de {{1312 | currency}}</p>
```

- Sans correction de locale :

le total de votre panier est de $1,312.00

- Avec correction de locale :

le total de votre panier est de 1 312,00 $US

- L'endroit où est affichée la devise et les séparateurs numériques sont adaptés, la monnaie reste le $. Un autre provider est nécessaire dans le **`app.ts`** pour passer en €:

```
{provide: DEFAULT_CURRENCY_CODE, useValue: 'EUR' }
```